

VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



**SEQUENTIAL DEEP LEARNING MODELS FOR PERSONALIZED
FASHION RECOMMENDATION**

By

Lê Huỳnh Nhã Nguyên

A thesis submitted to the School of Computer Science and Engineering

in partial fulfillment of the requirements for the degree of

Bachelor of Data Science

Ho Chi Minh City, Vietnam

2025-2026

SEQUENTIAL DEEP LEARNING MODELS FOR PERSONALIZED FASHION RECOMMENDATION

APPROVED BY:

Assoc. Prof. Dr. Nguyen Thi Thuy Loan, Chair

Assoc. Prof. Dr. Nguyen Van Sinh

Dr. Nguyen Thi Thanh Sang

THESIS COMMITTEE

ACKNOWLEDGMENTS

This work represents an extension of my efforts toward the final thesis required for the completion of my Bachelor's degree in Data Science, as stipulated by the School of Computer Science and Engineering. The success of this endeavor would not have been possible without the unwavering support and guidance of Dr. Nguyen Thi Thanh Sang, to whom I express my deepest gratitude. I am profoundly thankful for her willingness to serve as my supervisor from the outset of the Special Study phase within this field. Her insightful advice and direction have been instrumental in helping me navigate the complexities of this research, particularly during moments of uncertainty, enabling me to formulate a clear and structured approach to the thesis proposal.

I would also like to acknowledge the invaluable companionship of my closest friends, who have been a constant source of support since my initial days at university. Their encouragement during moments of self-doubt has been essential, and without their presence, my academic journey would have been far less fulfilling. I deeply value our friendship and the shared experiences of our youthful journey together.

The process of developing this proposal has been both intellectually enriching and personally rewarding, as it provides an opportunity to apply the knowledge I have acquired in a meaningful and impactful manner.

TABLE OF CONTENTS

ACKNOWLEDGMENTS	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	v
LIST OF TABLES.....	vi
ABBREVIATIONS	vii
ABSTRACT	viii
CHAPTER 1: INTRODUCTION.....	1
1.1. Background.....	1
1.1.1. E-commerce Recommendation Systems.....	1
1.1.2. Traditional Recommendation Approachs	1
1.1.3. Sequential Recommendation and Attention Mechanisms	2
1.2. Problem Statement.....	3
1.3. Scope and Objectives.....	4
1.4. Assumption and Solution.....	5
1.5. Structure of thesis	6
CHAPTER 2: LITERATURE REVIEW	8
2.1. Recommendation Systems.....	8
2.1.1 Introduction to Recommendation Systems.....	8
2.1.2. Categories of Recommendation Systems	9

2.2. Sequential Recommendation	11
2.3 Attention Mechanism & Transformer	13
2.3.1 Attention Mechanism	13
2.3.2 Self-Attention and Multi-Head Attention.....	14
2.3.3 Transformer Encoder	15
2.4 Attention-Based Models	17
2.4.1. Self-Attentive Sequential Recommendation (SASRec)	17
2.4.2. Bidirectional Encoder Representations from Transformers for Sequential Recommendation (BERT4Rec)	18
2.5. Neural Collaborative Filtering.....	20
CHAPTER 3: METHODOLOGY	23
3.1. General Idea and Framework	23
3.2. Data Collecting and Preprocessing.....	24
3.2.1 Data Collecting and Description.....	24
3.2.2 Data Preprocessing	25
3.3. Baseline Models and Proposed Enhanced Architecture	29
3.3.1 SASRec (Baseline Model).....	29
3.3.2 BERT4Rec (Baseline Model).....	30
3.3.3 Proposed Optimized BERT4Rec	30
3.3.4 Neural Collaborative Filtering (NCF)	34

3.4. Evaluation Metrics.....	36
3.4.1. Performance Evaluation of BERT4Rec.....	36
3.4.2. Performance Evaluation of NCF	37
CHAPTER 4: IMPLEMENTATION AND RESULTS	39
4.1. Implementation and Experiments.....	39
4.1.1. Implementation of the Proposed Optimized BERT4Rec	39
4.1.2. NCF Implementation	42
4.1.3. Demonstratinon	45
4.2. Results	51
4.2.1. Results of Sequential Recommendation Models	51
4.2.2. Results of the Neural Collaborative Filtering Model	54
CHAPTER 5: DISCUSSION AND EVALUATION.....	55
5.1. Discussion.....	55
5.2. Comparison.....	56
5.3. Evaluation.....	57
CHAPTER 6: CONCLUSION AND FUTURE WORK.....	59
6.1. Conclusion	59
6.2. Future Work.....	60
REFERENCES	62

LIST OF FIGURES

Figure 2. 1: Content-Based Filtering (CBF) Workflow [2].....	10
Figure 2. 2: User-Based Collaborative Filtering [2].....	11
Figure 2. 3: Item-Based Collaborative Filtering [2].....	11
Figure 2. 4: Unidirectional vs. bidirectional self-attention.....	14
Figure 2. 5: Scaled Dot-Product Attention and Multi-Head Attention [9].....	15
Figure 2. 6: The Transformer - Encoder.....	16
Figure 2. 7: Self-Attentive Sequential Recommendation.....	18
Figure 2. 8: BERT4Rec model architecture [11].....	19
Figure 2. 9: Neural collaborative filtering framework [15].....	22
Figure 3. 1: Overall Research Workflow of the Proposed Recommendation Framework.....	24
Figure 3. 2: Illustration of the Initial Data Structure (Sample Rows).....	25
Figure 3. 3: Architecture of the Proposed Optimized BERT4Rec model.....	31
Figure 3. 4: Architecture of the Neural Collaborative Filtering (NCF) model.....	34
Figure 4. 1: Main demonstration interface.....	46
Figure 4. 2: Visualization of the historical interaction sequence for a selected user retrieved	47
Figure 4. 3: Input interaction sequence and ground-truth next item	48
Figure 4. 4: Top-K recommendation results.....	48
Figure 4. 5: Product search interface for adding items to a custom interaction sequence.....	49
Figure 4. 6: Updated interaction sequence after appending new items	50
Figure 4. 7: Top-K recommendations generated from the modified sequence	50

LIST OF TABLES

Table 3. 1: Detailed Description of the Dataset Columns	25
Table 3. 2: Dataset statistics after preprocessing.....	26
Table 4. 1: Model Configuration and Hyperparameters for BERT-based Models.....	40
Table 4. 2: Model Configuration and Hyperparameters for NCF Models	44
Table 4. 3: Performance Comparison of Sequential Models.....	51
Table 4. 4: Performance Improvement of BERT4Rec+ over BERT4Rec	52
Table 4. 5: Performance Evaluation of the Neural Collaborative Filtering Model	54

ABBREVIATIONS

Notation	Full form
RS	Recommendation System
CF	Collaborative Filtering
CBF	Content-Based Filtering
NCF	Neural Collaborative Filtering
BERT4Rec	Bidirectional Encoder Representations from Transformers for Sequential Recommendation
SASRec	Self-Attention Sequential Recommendation
RNNs	Recurrent Neural Network
GRU	Gated Recurrent Unit
NDCG@K	Normalized Discounted Cumulative Gain at K
HIT@K	Hit Ratio at K
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
MAE	Mean Absolute Error
MLP	Multi-Layer Perceptron
MLM	Masked Language Modeling
DL	Deep Learning
API	Application Programming Interface
SDPA	Scaled Dot-Product Attention
FFN	Feed-Forward Network
LSTM	Long Short-Term Memory

ABSTRACT

In recent years, the data industry has experienced significant growth across all aspects of life, with Data Science becoming a key driver in many activities. E-commerce, in particular, leverages data flows effectively, making recommendation systems highly popular for enhancing customer experience and driving sales. These systems are crucial for personalizing user experiences, especially in the fashion and lifestyle beauty product domain, where modeling users' dynamic preferences from historical behaviors remains a challenging yet essential task.

This study focuses on improving sequential recommendation models, specifically BERT4Rec, by incorporating additional attributes and integrating a Neural Collaborative Filtering (NCF) model to assist with user rating prediction. We propose an optimized version of BERT4Rec, which addresses the limitations of unidirectional models by incorporating bidirectional self-attention mechanisms, thereby enhancing the contextual representation of user preferences. Furthermore, the NCF model, while not directly impacting the product ranking, improves the system's ability to predict how a user might rate products they have not interacted with.

To evaluate the effectiveness of the proposed model, we compare its performance with the original BERT4Rec and SASRec models on a beauty product dataset. Our experiments demonstrate that the optimized BERT4Rec, in combination with NCF, offers a more comprehensive and personalized recommendation system.

CHAPTER 1: INTRODUCTION

1.1. Background

1.1.1. E-commerce Recommendation Systems

Electronic commerce has experienced rapid growth, becoming one of the most influential components of the digital economy. As online platforms expand their catalogs to include vast and diverse product offerings, customers face a significant challenge in the decision-making process, requiring substantial time and effort to identify products aligned with their preferences. This phenomenon, often referred to as information overload, necessitates effective filtering tools.

Unlike traditional retail, e-commerce has the unique advantage of leveraging digital data to deliver personalized shopping experiences. With only a few interactions, such as product views, clicks, or search queries - online platforms can analyze user behavior and present items that are highly relevant to individual interests. This capability is largely enabled by recommendation systems, which assist users in making informed decisions, even when they lack detailed knowledge about the products they are evaluating.

From a business perspective, RS offers significant strategic value by increasing customer engagement, improving conversion rates, and enhancing user retention. In highly competitive digital markets, the ability to understand and anticipate dynamic user preferences is critical for sustained growth, making recommendation systems an indispensable tool for efficiently matching customer needs with suitable products.

1.1.2. Traditional Recommendation Approaches

Traditional recommended systems typically fall into three main categories: Collaborative Filtering (CF), Content-Based Filtering (CBF), and hybrid approaches. While these methods have

been widely adopted, they present several critical limitations, particularly in modern e-commerce environments. These methods often suffer from data sparsity and the cold-start problem. Crucially, they generally treat user-item interactions as a static matrix rather than a temporally ordered sequence. Consequently, they struggle to effectively capture the dynamic evolution of user preferences over time. These shortcomings motivate the subsequent development of sequential recommendation models, which explicitly model user behavior as an ordered sequence of interactions to better predict the next item a user is likely to engage with.

1.1.3. Sequential Recommendation and Attention Mechanisms

The primary goal of recommender systems is to model the compatibility between users and items based on their historical feedback, such as clicks, views, purchases, or ratings. In many real-world applications, these interactions are inherently sequential: they occur over time and are recorded in the order in which they happen. Consequently, a user's behavior history is viewed as a sequence of actions rather than an unordered set of events. Modeling these sequential dynamics is challenging because the input space grows rapidly as more past actions are used as context, and the dependencies between distant items in the sequence are often complex and nonlinear. For this reason, recent research increasingly focuses on sequential recommendation, in which user-item interactions are modeled as ordered sequences in order to predict the next item a user is likely to engage with.

Traditional sequence models, such as Recurrent Neural Networks (RNNs), often struggle to effectively capture long-range dependencies and to distinguish which past actions are truly important for the current prediction. Attention mechanisms address this limitation by dynamically computing a weight for each element in the sequence, indicating its relative importance, and aggregating these representations into a more informative context vector.

Building on this idea, self-attentive sequential recommendation employs self-attention to directly model the relationships among items within a user’s interaction sequence. This allows the model to better capture both short-term and long-term user preferences. This self-attentive paradigm forms the basis for advanced architectures such as SASRec and BERT4Rec, which leverage multi-head self-attention to learn rich representations of user behavior for the next-item recommendation task.

1.2. Problem Statement

Despite the rapid growth of e-commerce platforms and the widespread adoption of recommended systems, providing accurate and personalized next-item recommendations in highly competitive domains such as beauty products remains a challenging task. Traditional recommendation methods, as well as early deep learning-based models, often fail to capture the sequential nature of user behavior and the dynamic evolution of user preferences over time. Recent sequential models like SASRec and BERT4Rec address this issue by leveraging self-attention mechanisms; however, they are typically trained on interaction sequences without fully exploiting additional item attributes that may be highly informative in practice.

In particular, the original BERT4Rec model focuses on learning bidirectional representations from user-item interaction sequences but does not explicitly incorporate side information such as product type, which is crucial in domains with strong category-specific preferences like fashion and lifestyle beauty products. Moreover, most existing sequential recommendation models are primarily designed to generate ranked lists of items without integrating a dedicated rating prediction component that could provide further insight into how users might evaluate unrecommended products.

Therefore, there is a need for an enhanced sequential recommendation framework that improves the representation of user behavior by incorporating meaningful item attributes and supplements the recommendation process with a rating prediction module. This study addresses this problem by optimizing the BERT4Rec model with additional product type information and integrating a Neural Collaborative Filtering (NCF) component to predict user ratings for candidate items in the beauty product domain, and by empirically comparing its performance against SASRec and the original BERT4Rec.

1.3. Scope and Objectives

This study investigates self-attentive sequential recommendation models in the fashion and lifestyle beauty product domain, with a particular focus on optimizing the BERT4Rec architecture. The experiments are conducted on a beauty product dataset containing user–item interaction histories along with basic item attributes such as product type. The proposed approach does not incorporate additional data modalities, including textual reviews, images, or social network information, nor does it address system-level challenges such as large-scale deployment or real-time inference. A lightweight prototype is implemented solely for demonstration purposes, aiming to illustrate the inference process and the practical usage of the proposed recommendation models, rather than serving as a fully deployed or production-ready system.

The objectives of this thesis are as follows:

- To implement baseline sequential recommendation models, including SASRec and the original BERT4Rec, on a beauty product dataset containing items.
- To propose and develop an optimized version of BERT4Rec that explicitly incorporates product type as an additional item attribute in the user-item interaction sequences in order to better capture contextual user preferences.

- To integrate a Neural Collaborative Filtering (NCF) model as an auxiliary component for predicting user ratings on candidate items that have not yet been recommended, thereby enriching the recommendation output with rating information.
- To conduct an empirical comparison of SASRec, the original BERT4Rec, and the optimized BERT4Rec combined with NCF using standard recommendation metrics, in order to assess their effectiveness in terms of accuracy and personalization in the beauty e-commerce scenario.
- To demonstrate an interactive prototype of the proposed recommendation framework that provides top-K beauty product recommendations together with predicted ratings.

1.4. Assumption and Solution

In this thesis, several assumptions are made to facilitate the design of the recommendation system based on BERT4Rec and Neural Collaborative Filtering (NCF). First, it is assumed that user interactions with products follow a sequential pattern, where recent behavior significantly influences the next item a user is likely to engage with. It is also assumed that the ratings in the dataset reflect the strength of user preferences, and that ProductType provides valuable insights for distinguishing between different categories of items, which will improve the model’s ability to recommend relevant products. The dataset is assumed to be sufficiently large and clean for training both models. Regarding evaluation, we assume that using a sampled protocol with 1 true item and 100 random negatives is a valid method to assess BERT4Rec’s performance. Metrics such as Hit@K and NDCG@K are considered appropriate for measuring ranking quality in this context.

The proposed solution involves the development of a sequential recommendation framework centered on optimizing the BERT4Rec architecture. This optimized model is used to generate a top-K list of products by leveraging token, positional, and product type embeddings to enhance

contextual understanding. The top-K candidates are then processed by an auxiliary Neural Collaborative Filtering (NCF) model, which predicts user ratings for these items. While initial experiments indicated that using NCF for reranking did not significantly improve standard ranking metrics, the primary purpose of the integrated NCF component is to provide predicted rating information to the user interface as supplementary data. The effectiveness of the overall solution is assessed by empirically comparing the Optimized BERT4Rec against baseline models, including the original BERT4Rec and SASRec.

1.5. Structure of thesis

This thesis is structured into the following chapters, each contributing to a comprehensive understanding of the proposed recommendation system:

Introduction: This chapter introduces the background of the study, outlining the research objectives, the major goals of the project, and the overall scope of the thesis. It sets the stage for the subsequent chapters and provides context for the importance of the research.

Literature Review: This chapter reviews the core techniques and models employed in this study, including BERT4Rec for sequential recommendations and Neural Collaborative Filtering (NCF) for rating prediction. It provides a detailed overview of the existing literature on sequential recommendation systems.

Methodology: In this chapter, the methodology of the study is described in detail. It explains the data flow during the training phase, the approach used to apply BERT4Rec together with an auxiliary NCF component, and how these models are utilized to generate product recommendations and predict ratings. This chapter also highlights the steps taken to enhance the recommendation process and improve overall system performance.

Implementation and Results: This chapter presents the experimental setup and results from the implementation of BERT4Rec and NCF. It details the evaluation metrics used and compares the performance. The findings from these experiments are analyzed and discussed to highlight the strengths and areas for improvement.

Discussion and Evaluation: This chapter provides an in-depth discussion of the results obtained from the experiments. It evaluates the effectiveness of the proposed system, reflecting on its advantages and limitations, and compares it with existing approaches. The chapter also assesses the practical applicability and reliability of the system in real-world recommendation scenarios.

Conclusion and Future Work: The final chapter summarizes the key findings of the research and discusses potential directions for future work, suggesting improvements that could further enhance the system's performance and adaptability across different recommendation domains..

CHAPTER 2: LITERATURE REVIEW

2.1. Recommendation Systems

2.1.1 Introduction to Recommendation Systems

In the modern digital economy, consumers are constantly confronted with an overwhelming abundance of choices across nearly every domain, from content consumption to product purchasing. This phenomenon often leads to information overload, making the decision-making process challenging and time-consuming for users. To mitigate this issue, Recommendation Systems (RS) have become essential tools, functioning as information filtering systems designed to guide users toward content or products that align with their specific preferences and interests. These systems operate by leveraging rich streams of historical user data, such as clicks, purchase histories, and browsing patterns to predict and suggest items most likely to resonate with the individual.

A Recommendation System (RS) is an information filtering system designed to predict the "rating" or "preference" a user may assign to an item, making it an indispensable tool in various digital platforms [1]. The main objective of an RS is to model the relationship between users and items, filtering and narrowing down the vast number of options to a personalized list of suggestions [1]. This process enhances user experience by offering tailored recommendations based on previous interactions with the platform.

RS are widely used across multiple domains, including online shopping platforms (such as Amazon, which suggests products based on browsing and purchase history) [3], music streaming services (Spotify, which recommends songs based on listening behavior), and streaming platforms (Netflix, which suggests movies and TV shows based on viewing patterns) [3]. By helping users discover relevant content, recommendation systems improve e-commerce performance, increase

platform engagement, and drive sales. In essence, RS have become a core component of electronic commerce, enhancing the user experience and contributing to platform success.

2.1.2. Categories of Recommendation Systems

Recommendation Systems (RS) are generally classified into several categories, each utilizing different approaches to generate tailored recommendations. The three main categories of recommendation systems are Content-Based Filtering (CBF), Collaborative Filtering (CF), and Hybrid Systems [1].

2.1.2.1. Content-Based Filtering

Content-Based Filtering is a recommendation method that suggests items based on their intrinsic features and the user's past interactions [2]. It analyzes the characteristics of items that the user has shown interest in, such as genres, keywords, or specific attributes, and recommends similar items. The idea is simple: if a user likes an item with certain features, they are likely to enjoy other items that share the same characteristics.

One major advantage of content-based filtering is that it does not rely on data from other users, making it particularly useful in privacy-sensitive contexts like healthcare or finance. It also helps solve issues related to data sparsity and cold start [1], as recommendations are made based on the individual's own preferences. However, the effectiveness of this approach depends heavily on the quality of the item features [2]. If the item attributes are poorly defined or fail to capture the full complexity of user preferences, the recommendations may not be optimal. Therefore, content analysis is crucial to ensure the features are well-defined and truly representative of user interests.

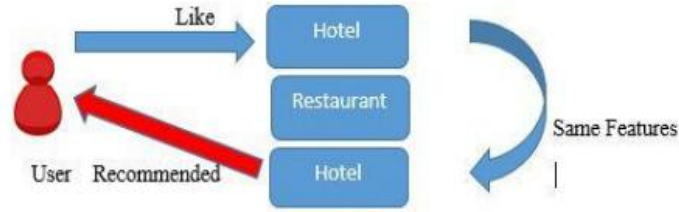


Figure 2. 1: Content-Based Filtering (CBF) Workflow [2]

2.1.2.2. Collaborative Filtering

Collaborative Filtering (CF) is a method that determines an individual user's preferences by analyzing and aggregating the interaction patterns of a wider group of users who exhibit similar tastes and behaviors [1]. Instead of relying on item attributes, CF focuses on how users interact with items - such as views, purchases, clicks, or ratings - and identifies similarities either between users or between items. The underlying assumption is that users who behaved similarly in the past will likely share preferences in the future.

There are two main forms of collaborative filtering. User-based CF identifies groups of users with similar interaction histories and recommends items that peers with similar tastes have liked [2]. For example, if two users consistently rate or interact with the same items, the system may recommend an item preferred by one user to the other. In contrast, item-based CF examines the relationships between items by analyzing how users rate or interact with them [2]. Items frequently rated or consumed together are considered similar, and the system recommends new items based on these item-item correlations.

One advantage of collaborative filtering is that it can capture complex user interests through collective behavior rather than requiring item features. However, CF suffers from limitations such as the cold-start problem, where new users or items lack sufficient interaction data to generate accurate recommendations. Additionally, its performance can degrade in sparse datasets, where user-item interactions are limited [2].

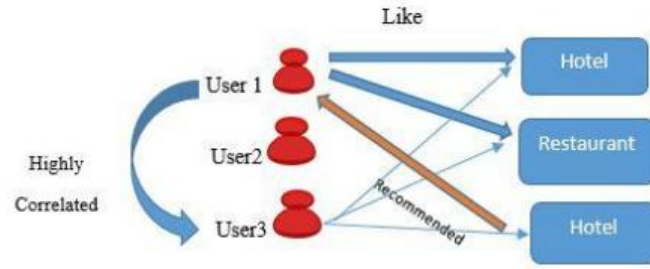


Figure 2. 2: User-Based Collaborative Filtering [2]

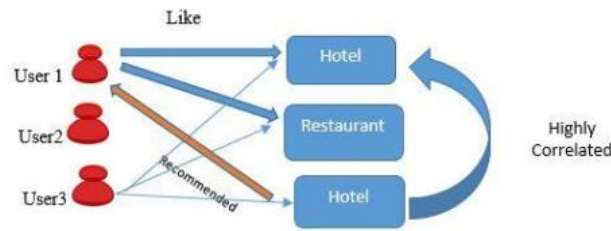


Figure 2. 3: Item-Based Collaborative Filtering [2]

2.1.2.3. Hybrid Models

Hybrid recommender systems combine two or more recommendation techniques to overcome the limitations of individual methods and improve overall performance [1]. A common hybrid form integrates content-based filtering and collaborative filtering, allowing the system to leverage item features while also capturing patterns from user-item interactions. This combination helps address issues such as cold-start, sparsity, and limited personalization.

Hybrid systems may blend model outputs, augment features, or cascade multiple algorithms to generate more accurate and robust recommendations. By utilizing complementary strengths, hybrid approaches provide more stable and reliable predictions compared to using a single method alone.

2.2. Sequential Recommendation

Sequential Recommendation (SR) is a specialized area of study focused on predicting user preferences by explicitly exploiting the internal order and dependency among the interacted items [4]. Unlike traditional recommendation methods that treat user-item interactions as an unordered

set, sequential approaches emphasize the importance of action order, as the most recent activities often provide the strongest signals about a user’s evolving preferences.

Early studies in this area typically relied on Markov Chain (MC) techniques, which model the probability of transitioning from one item to the next. First-order MCs assume that the next interaction depends solely on the immediately previous item, while higher-order variants incorporate a longer history of past actions. Although these methods are simple and effective on short sequences, their ability to capture complex long-range dependencies is limited [5].

To overcome the limitations of Markov Chain-based approaches, subsequent work introduced Deep Learning (DL) models for sequential behavior modeling. Recurrent Neural Networks (RNNs), particularly Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM) architectures, became widely adopted. [6,7]. These models are valued for their capacity to encode the entire user interaction sequence into a single latent representation, thereby capturing temporal dependencies directly from user histories. For instance, models like GRU4Rec demonstrated strong performance in session-based recommendation.

However, RNN-based models have inherent constraints. Despite their ability to capture short- and medium-term patterns, they are fundamentally limited in modeling long-range dependencies due to the sequential nature of their computation [7]. Furthermore, standard RNN architectures may struggle to filter and manage the limited number of input variables.

Beyond RNNs, other neural architectures have been explored to address the limitations of sequential computation. Convolution-based models, such as Caser (Context-Aware Sequential Recommendation), leverage convolutional filters to extract sequential patterns directly from item interaction embeddings [8]. Caser's approach, which applies horizontal and vertical convolutional filters, is designed to capture both short- and long-range transitions in user behavior effectively. Nonetheless, these limitations ultimately motivated the research transition toward more

parallelizable and context-aware architectures, such as Self-Attention, which directly models dependencies regardless of their distance in the sequence.

2.3 Attention Mechanism & Transformer

2.3.1 Attention Mechanism

The attention mechanism has recently gained significant attention in sequential modeling tasks, particularly in fields such as machine translation and natural language processing. Its strength lies in its ability to focus on the most relevant parts of an input sequence, enabling models to capture long-range dependencies that are challenging for traditional models like RNNs and LSTMs [9]. Unlike conventional approaches, where the model may struggle to understand distant relationships in the data, attention allows the model to selectively prioritize parts of the input that are more informative, thereby improving both accuracy and efficiency.

In sequential recommendation, the use of Self-attention mechanisms plays a crucial role by allowing the model to capture not only short-term preferences (such as the most recent interactions) but also long-term dependencies across user behavior sequences. This is particularly important in recommended systems, where understanding the full context of a user’s interaction history enables more accurate next-item predictions. The ability to model both past and future interactions in a bidirectional manner leads to a richer understanding of user preferences.

Traditional self-attentive sequential models, such as SASRec, are limited by their use of a unidirectional attention mechanism [4,10]. This architectural constraint means the model can only consider preceding interactions in the sequence, thereby restricting its ability to capture the full context and impacting the quality of recommendations.

In contrast, BERT4Rec addresses this major limitation by leveraging a bidirectional self-attention mechanism [11]. The bidirectional approach enables the model to learn from both

preceding and subsequent interactions in the user's history, leading to a richer understanding of dynamic and complex user preferences over time [12]. This capability allows BERT4Rec to capture more nuanced user behaviors and ultimately yields predictions with higher accuracy, significantly improving the recommendation process.

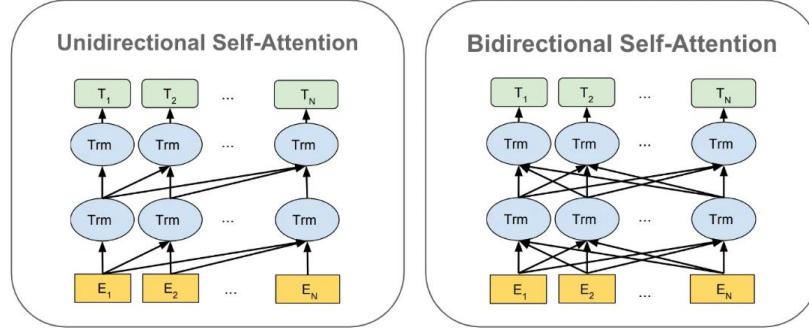


Figure 2. 4: Unidirectional vs. bidirectional self-attention

2.3.2 Self-Attention and Multi-Head Attention

In sequential recommendation models, the concept of Self-Attention is extended through the use of Multi-Head Attention. Instead of relying on a single attention function, this mechanism employs multiple parallel attention layers, or 'heads,' to capture different aspects of the input sequence [9]. Multi-Head Attention allows the model to jointly attend to information from different representation subspaces at different positions, preventing a single head from inhibiting the model's ability to extract diverse features [9]. The outputs from these heads are then concatenated and linearly combined to create a comprehensive and diverse context vector. This parallel processing mechanism significantly improves the model's capacity to extract diverse features and learn richer representations for user-item interactions.

The core mechanism used by each attention head is the Scaled Dot-Product Attention (SDPA). SDPA is the attention function employed within the Transformer architecture and operates through a complex matrix calculation [9]. This mechanism determines attention weights by taking the dot

products of the query (Q) and all keys (K), dividing each product by the scaling factor $\sqrt{d_k}$ (to prevent large dot products from overloading the SoftMax function), and then applying the Softmax function to obtain the weights on the values (V).

Mathematically, the SDPA function is defined as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad [9]$$

In BERT4Rec, the potent combination of bidirectional self-attention and Multi-Head Attention enables the model to effectively capture dependencies across the entire sequence of user behavior. This powerful approach contrasts with models like SASRec, which use unidirectional attention and are consequently limited to considering only preceding interactions in the history. By learning from both directions, BERT4Rec provides a richer and more complete contextual understanding of user preferences, leading to more accurate predictions.

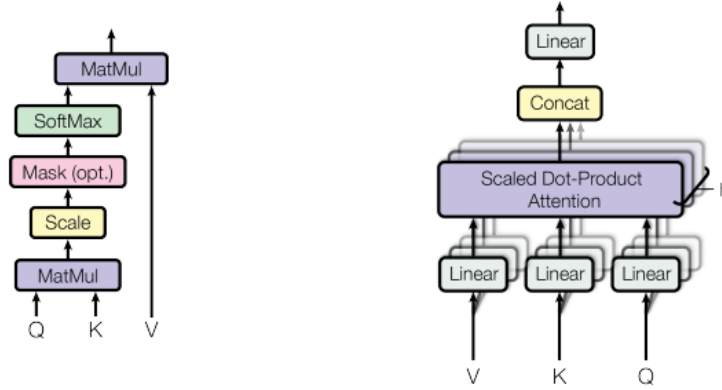


Figure 2. 5: Scaled Dot-Product Attention and Multi-Head Attention [9]

2.3.3 Transformer Encoder

The Transformer Encoder is the key component in the Transformer architecture, introduced by Vaswani *et al.* (2017) [9], which revolutionized sequence-based tasks. Unlike traditional recurrent models (RNNs or LSTMs), the Encoder uses self-attention to simultaneously process all positions

in the input sequence. This eliminates the need for sequential computation, making the architecture highly parallelizable and more efficient.

The Encoder is typically composed of several identical layers. Each layer consists of two primary components: a Multi-Head Self-Attention (MHSA) mechanism and a Position-wise Feed-Forward Network (FFN) [9]. The MHSA layer is responsible for learning different representations of the input sequence by attending to various parts simultaneously. After the MHSA output is processed, it is passed through the FFN, which applies non-linear transformations independently to each position in the sequence.

Furthermore, each sub-layer in the Encoder integrates Residual Connections and is followed by Layer Normalization to stabilize training and facilitate gradient flow [13]. Since the Transformer inherently lacks recurrence, Positional Encoding is added to the input embeddings to inject information about the sequence order, which is crucial for sequential modeling tasks.

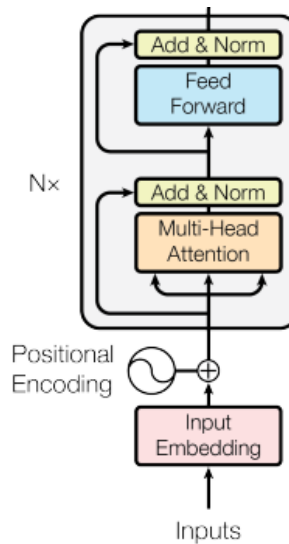


Figure 2. 6: The Transformer - Encoder

2.4 Attention-Based Models

Attention-based models have become central to modern sequential recommendation systems due to their ability to selectively focus on the most relevant interactions in a user's history. Several variants of attention mechanisms have been explored in prior work, such as Time Interval Aware Self-Attention for Sequential Recommendation (TiSASRec), which incorporates temporal intervals into attention to enhance sequential recommendation [14]. These models demonstrate the flexibility of attention mechanisms in capturing user behavior patterns. Building upon this foundation, two of the most influential architectures in sequential recommendation, SASRec and BERT4Rec, have been extensively developed and applied across various domains of recommendation systems.

2.4.1. Self-Attentive Sequential Recommendation (SASRec)

SASRec is a foundational self-attention-based model for sequential recommendation that adopts pure Transformer-style architecture to model user interaction sequences [10]. The model represents each item through learnable embeddings and incorporates Positional Encoding to preserve the order of interactions. This is essential since pure Transformer architecture lacks recurrence, which would otherwise lead to a loss of sequence information.

SASRec utilizes a series of Transformer Encoder blocks, where each block consists of a Multi-Head Self-Attention layer followed by a point-wise Feed-Forward Network. Crucially, the attention mechanism employed is the Scaled Dot-Product Attention (SDPA) [4, 10]. However, the model implements this mechanism with a causality mask to enforce a unidirectional self-attention flow. This design ensures that the prediction for the next item is based only on preceding interactions, selectively emphasizing relevant past behavior patterns while naturally filtering out future signals.

Because of its simplicity, efficiency, and strong empirical performance, SASRec has become a widely used baseline in the evaluation of more advanced sequential models. Its unidirectional design also provides a natural and necessary contrast to later architectures, such as BERT4Rec, which extend the self-attention mechanism by incorporating bidirectional contextual modeling.

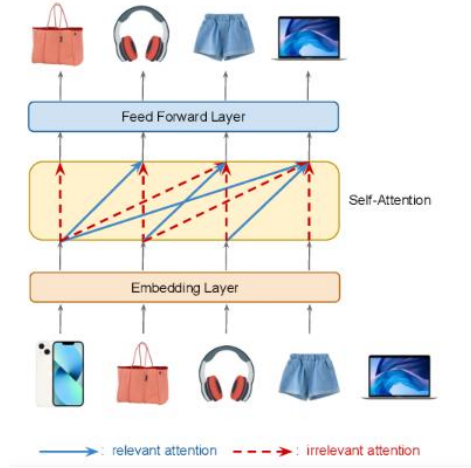


Figure 2. 7: Self-Attentive Sequential Recommendation

2.4.2. Bidirectional Encoder Representations from Transformers for Sequential Recommendation (BERT4Rec)

BERT4Rec, introduced by *Sun et al.*, represents a significant advancement in attention-based sequential recommendation by extending the bidirectional modeling capabilities of BERT by *Devlin et al.*, to user-item interaction sequences [11, 12]. Unlike earlier self-attentive models that rely on unidirectional encoding, BERT4Rec adopts a bidirectional Transformer encoder to capture contextual relationships from both preceding and succeeding items within a sequence [4, 11, 12]. This architectural shift is motivated by the observation that user preferences are often shaped by complex global dependencies that cannot be fully leveraged when attention flows only from left to right. By transferring the principles of masked language modeling to the recommendation domain, BERT4Rec aims to exploit the full contextual information embedded in user histories, thereby enabling richer sequential representations and more accurate next-item predictions.

A central innovation of BERT4Rec lies in its use of bidirectional self-attention to model user-item interaction sequences. To train such a bidirectional architecture without leaking future information, the model adopts a Cloze-style Masked Item Prediction (MIP) objective inspired by the masked language modeling task used in BERT [11, 12]. Specifically, a random subset of items in the input sequence is replaced with a special mask token, and the model is required to recover the original items based solely on the surrounding left and right context. This learning objective allows BERT4Rec to capture global dependencies more effectively than unidirectional sequential models, while simultaneously generating multiple training samples from a single sequence. Its loss function is formally defined following the negative log-likelihood over the masked positions in following equation.

I see that you've uploaded an image containing a mathematical expression. I can help rewrite it for you. The expression is:

$$\mathcal{L} = \frac{1}{|S_u^m|} \sum_{v_m \in S_u^m} -\log P(v_m = v_m^* | S_u')$$

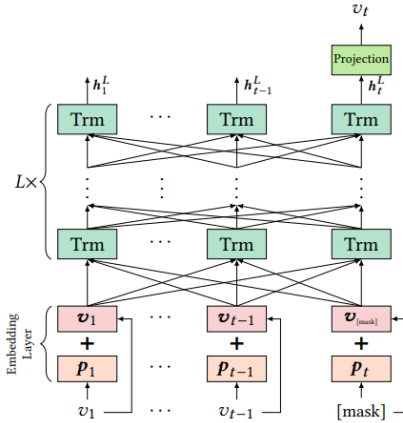


Figure 2. 8: BERT4Rec model architecture [11]

In BERT4Rec, each input item v_t is projected into a continuous vector space and combined with a learnable positional embedding p_t , forming the input representation $h_t^0 = v_t + p_t$ at the

bottom of the encoder stack. The sequence is then processed through L Transformer layers, each composed of a multi-head self-attention sublayer and a position-wise feed-forward network, enabling the model to aggregate contextual information from all positions in parallel [11]. During training, a subset of items is randomly masked, and the model predicts the masked targets using a Cloze-style objective, where the final hidden state corresponding to a masked position is fed into a shared projection layer to compute scores via the inner product with output item embeddings. The predicted distribution is optimized through a cross-entropy loss over all masked positions, allowing the model to jointly condition on left and right context while benefiting from parallel computation across the entire sequence. An illustration of model architecture is shown in Figure 2.8.

Its ability to leverage full bidirectional context and generate multiple training samples from a single sequence makes it an effective and stable foundation for subsequent research in sequential recommendation. Following its introduction, numerous studies have expanded upon BERT4Rec by proposing improved masking strategies, incorporating richer contextual or side information, and integrating contrastive learning objectives to enhance representation learning. Despite its empirical advantages, BERT4Rec still presents several practical limitations, including a relatively high training cost and reduced efficiency on very long sequences, which has motivated the development of lighter or more scalable Transformer-based architectures. Overall, BERT4Rec has become a pivotal reference model in the literature, shaping the direction of recent advances in attention-based sequential recommendation.

2.5. Neural Collaborative Filtering

Collaborative Filtering (CF) methods, especially Matrix Factorization (MF), have been widely adopted to model user preferences by projecting users and items into a shared latent space and

computing their interaction via inner product [16, 17]. However, the linear nature of the inner product limits MF’s ability to capture complex, non-linear user-item interactions. To overcome this limitation, recent research has turned to Deep Neural Networks (DNNs) [15], which are capable of learning highly complex, non-linear functions from data.

Motivated by these advances, Neural Collaborative Filtering (NCF) has been proposed as a deep learning-based recommendation framework that replaces the simple inner product in MF with a neural network-based interaction function [15]. By learning the interaction function directly from data, NCF enables the model to capture complex and non-linear feature interactions between users and items. This design enhances the expressive power of recommendation models, bridging the gap between traditional MF approaches and modern deep learning techniques.

Neural Collaborative Filtering (NCF) is a class of models that leverages neural networks to learn complex, non-linear interactions between users and items for recommendation tasks [15, 18]. NCF addresses the limitations of traditional matrix factorization methods by replacing inner products with neural network architectures that can better model high-order feature interactions.

At the heart of NCF is an embedding layer to map each user $u \in U$ and each item $i \in I$ into corresponding feature vectors $e_u \in \mathbb{R}^d$ and $e_i \in \mathbb{R}^d$. These vectors capture the latent characteristics of users and items, and they are subsequently used as inputs to a multi-layer perceptron (MLP) network [15]. Unlike traditional matrix factorization models that rely on the dot-product between vectors, NCF combines the user and item embeddings by concatenating them:

$$x_{ui} = [e_u \parallel e_i] \text{ [15]}$$

This concatenated vector is then passed through a series of fully connected layers to model the interactions between the user and the item. The prediction model of NCF can be represented as:

$$\hat{y}_{ui} = f(P^T v_u, Q^T v_i \mid P, Q, \Theta_f) \text{ [15]}$$

In this model, P and Q represent the latent factor matrices for users and items, respectively. The vectors v_u and v_i correspond to the latent vectors for a given user u and item i . Additionally, Θ_f refers to the parameters of the neural network f , which is implemented as a multi-layer perceptron (MLP) to capture and model the complex interactions between users and items [15]. To train the NCF model, a binary cross-entropy loss function is used, combined with negative sampling.

$$\mathcal{L}_{BCE} = - \sum_{(u,i) \in D^+ \cup D^-} [y_{ui} \log(\hat{y}_{ui}) + (1 - y_{ui}) \log(1 - \hat{y}_{ui})] \quad [15]$$

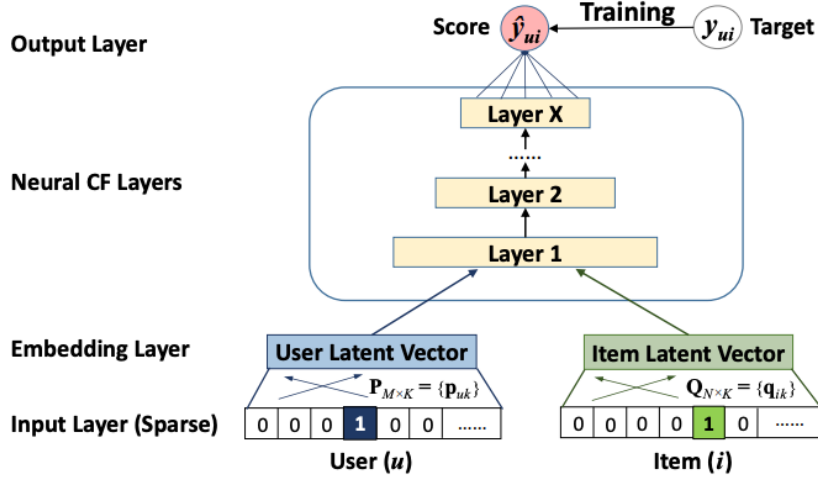


Figure 2. 9: Neural collaborative filtering framework [15]

NCF has significantly advanced collaborative filtering by introducing neural architectures capable of modeling complex user-item interactions beyond linear assumptions. Its flexibility and extensibility have inspired a wide range of follow-up models, bridging deep learning with personalized recommendation systems. Despite its strength, NCF still faces challenges in scalability, cold-start problems, and incorporating sequential or contextual signals, which are often addressed by combining NCF with other techniques such as self-attention or knowledge graph embeddings.

CHAPTER 3: METHODOLOGY

3.1. General Idea and Framework

The general idea of this study is to develop a sequential recommendation system that leverages both established baseline models and an enhanced Transformer-based architecture to improve next-item prediction performance in the fashion and lifestyle beauty product domain. The research begins by reproducing two widely used sequential recommendation models - SASRec and the original BERT4Rec - following their respective papers. These baseline models serve as reference points to ensure that the evaluation of the improved model is fair, reproducible, and scientifically grounded.

Building upon these baselines, the thesis proposes an optimized version of BERT4Rec in which an additional feature embedding is incorporated into the item representation. Enriching BERT4Rec with category-level semantic information is expected to enhance recommendation quality. This optimized model is trained and compared against the two baselines to verify whether the additional feature contributes to meaningful improvements in sequential recommendation performance.

To complement the sequential recommendation stage and enhance the interpretability of the final output, a Neural Collaborative Filtering (NCF) model is trained separately to predict user-item ratings. Although NCF is not used to re-rank the recommendation list, its predicted rating scores provide useful feedback to users and improve the overall user experience.

Finally, the proposed research framework is demonstrated through an interactive prototype designed as a proof-of-concept to visualize recommendation results and illustrate how the trained models operate in practice. The optimized BERT4Rec model generates top-K candidate items, while the NCF model predicts corresponding rating scores, which are then presented together to users through the demonstration system.

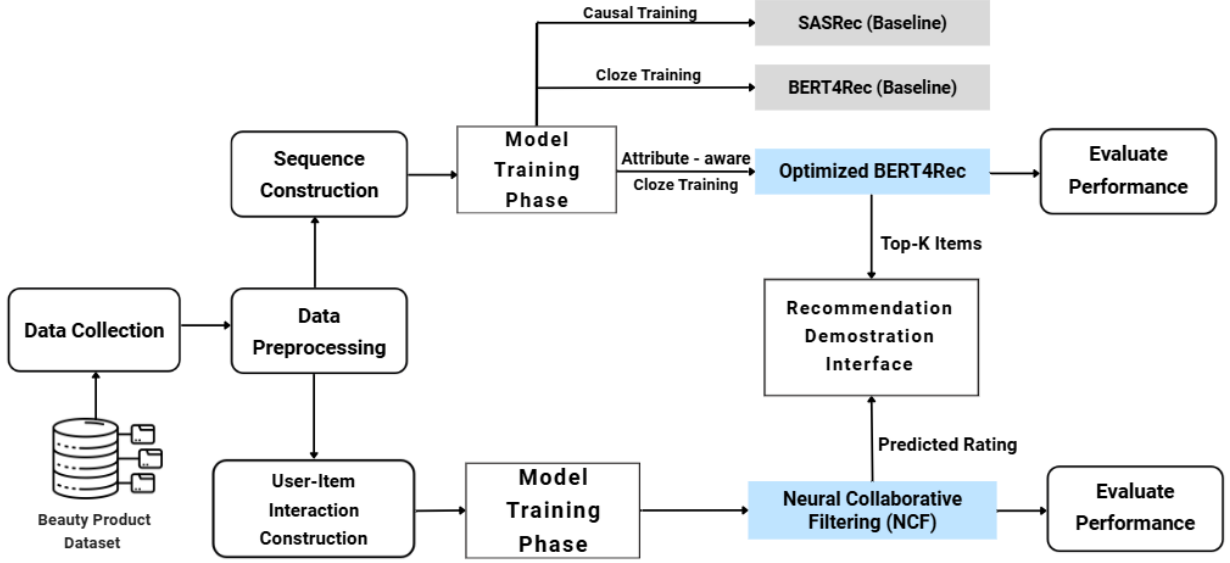


Figure 3. 1: Overall Research Workflow of the Proposed Recommendation Framework

3.2. Data Collecting and Preprocessing

3.2.1 Data Collecting and Description

The dataset used in this study was collected from the Amazon Beauty category, which is publicly available on Kaggle X . Each entry corresponds to a user's interaction with a beauty product and includes key attributes such as UserId, ProductId, ProductType, Rating, Timestamp, and the product URL.

Since the original dataset does not explicitly provide product brand information, an additional preprocessing step was applied to extract the brand name from the product URL. The extracted brand attribute was added as a new column to enrich item metadata and support result presentation in the recommendation demonstration interface.

UserId	ProductId	ProductType	Rating	Timestamp	URL	Brand
A3NHUQ33CFH3VM	B00LLPT4HI	Eyeliners & Kajal	5	1405814400	https://www.amazon.in/Maybelline	MAYBELLINE
A2Y36BR4YSY9F7	B00LLPT4HI	Eyeliners & Kajal	5	1405728000	https://www.amazon.in/Maybelline	MAYBELLINE
A2BLFCOPSMBOZ9	B00LLPT4HI	Eyeliners & Kajal	5	1405468800	https://www.amazon.in/Maybelline	MAYBELLINE
A34BZM6S9L7QI4	B00LLPT4HI	Eyeliners & Kajal	5	1405641600	https://www.amazon.in/Maybelline	MAYBELLINE
A3LGT6UZL99IW1	B00LLPT4HI	Eyeliners & Kajal	5	1405382400	https://www.amazon.in/Maybelline	MAYBELLINE
A3UJRN8UR4871	B00LLPT4HI	Eyeliners & Kajal	5	1405987200	https://www.amazon.in/Maybelline	MAYBELLINE

Figure 3. 2: Illustration of the Initial Data Structure (Sample Rows)

Table 3. 1: Detailed Description of the Dataset Columns

Column Name	Data Types	Description
UserId	Object	Unique identifier of a user who interacted with a beauty product
ProductId	Object	Unique identifier of a product in the Amazon Beauty catalog
ProductType	Object	The product's category
Rating	Int64	Rating score given by the user (ranging from 1 to 5)
Timestamp	Int64	Unix timestamp representing when the interaction occurred
URL	Object	URL link to the product's page on Amazon
Brand	Object	Brand name extracted from the product URL during the preprocessing stage.

3.2.2 Data Preprocessing

The raw dataset was first loaded from the Kaggle Amazon Beauty CSV file. Basic data cleaning steps were applied to remove records containing null, missing, or invalid values in essential fields such as user identifiers, product identifiers, ratings, and timestamps.

Since the original dataset does not explicitly include a product brand attribute, an additional preprocessing step was applied to extract the brand name directly from the product URL. The extracted brand attribute was stored in a new column and used to enrich item metadata for result presentation in the recommendation demonstration interface.

A basic filtering step was then performed to remove noisy and incomplete interaction records, ensuring that only valid user-item interactions were retained for subsequent analysis. After this general preprocessing stage, the dataset contains 27,485 unique users and 22,129 unique items.

After preprocessing, the key statistics of the dataset are summarized in Table X, including the number of users, items, interactions, and average sequence length.

Table 3. 2: Dataset statistics after preprocessing

Number of users	Number of items	Number of interactions	Average sequence length
27,485	22,129	0.24M	8.6

3.2.2.1 Preprocessing for Sequential Recommendation Models

The preprocessing procedure is implemented through a dedicated data processing pipeline, which converts raw user-item interaction records into sequential data suitable for Transformer-based recommendation models. This process is designed to prepare input sequences for the proposed Optimized BERT4Rec model and consists of the following steps:

Base sequence preprocessing includes the following steps:

- ID Encoding (User & Product): raw user and product identifiers were transformed into continuous integer indices using dictionary-based mappings. Each unique product was mapped to an item index, with index 0 strictly reserved for padding to maintain sequence consistency. User identifiers were encoded solely to enable chronological grouping of interactions into individual user sequences and were not used as direct inputs for representation learning within the Transformer architecture.
- Product-Type Feature Encoding: product categories were extracted and assigned stable numerical indices. These categorical indices were subsequently transformed into dense

representations via a dedicated type embedding layer within the model, allowing product-type information to serve as auxiliary semantic features alongside item embeddings.

- **Sequence Construction and Data Splitting:** interaction records were sorted by user identifiers and timestamps to form chronological behavior sequences. A leave-one-out evaluation strategy was adopted, where the final interaction in each sequence was used as the test target, the penultimate interaction as the validation target, and the remaining interactions as the training sequence. A parallel product-type sequence was constructed to ensure alignment with the item sequence.

During training, model-specific data generation is performed for Optimized BERT4Rec models as follows:

- **Masked Language Modeling (MLM):** base interaction sequences were dynamically transformed into masked sequences to support the BERT-style self-supervised learning objective. A fixed masking probability was applied to randomly mask items within each sequence, and the model was trained to predict the original items at the masked positions based on surrounding contextual information. For each user interaction sequence, each item is selected for masking with a predefined probability p . When an item is chosen for masking, one of the following replacement strategies is applied:

80% of the time, the original item is replaced with a designated special token, while the corresponding product-type feature is simultaneously replaced with its associated special type token to preserve feature alignment.

10% of the time, the item is substituted with a randomly sampled item, while its associated product type is updated accordingly to maintain feature consistency.

10% of the time, the original item and its product type are retained without modification.

- **Padding and Truncation:** to ensure compatibility with the Transformer architecture, all input sequences were standardized to a fixed maximum length. Specifically, item interaction sequences, product-type sequences, and the corresponding label sequences were either padded or truncated to achieve uniform sequence lengths. For sequences shorter than maximum length, padding tokens were appended to fill the remaining positions, while for sequences exceeding maximum length, only the most recent interactions were retained and earlier items were truncated. This design preserves the temporal relevance of user behavior while maintaining consistent input dimensions across all samples.

The sequential models under analysis share core sequence-level preprocessing steps, including ID encoding and chronological sequence construction, to standardize the interaction data. Input length standardization through padding and truncation is applied during model training to ensure consistent input dimensions. However, notable divergence exists concerning feature inputs and training strategies. Both the SASRec and original BERT4Rec baselines utilize only item ID sequences as model inputs and do not incorporate product-type information. Furthermore, SASRec does not employ masked language modeling, as it relies on a traditional autoregressive next-item prediction objective. Consequently, Optimized BERT4Rec is the sole model to leverage the complete preprocessing pipeline, incorporating both the product-type vector sequence and MLM. This controlled variation in the pipeline ensures that observed performance differences can be accurately attributed to the integration of auxiliary features and architectural modifications introduced in the Optimized BERT4Rec model.

3.2.2.2 Preprocessing for NCF

This section describes the preprocessing steps applied to prepare the user-product interaction data for the Neural Collaborative Filtering (NCF) model.

- **ID Encoding:** the raw `UserId` and `ProductId` fields were transformed into continuous integer indices through dictionary-based mappings. Each unique user and product was assigned a corresponding index, enabling the model to embed these identifiers effectively. This step standardizes categorical IDs into numerical representations suitable for neural network-based rating prediction models.
- **Tensor Conversion of Interaction Data:** after encoding, all user IDs, product IDs, and rating values were converted into PyTorch tensors. This ensures compatibility with GPU-accelerated training and allows efficient batching. The resulting tensors represent the fundamental input components of the NCF model - user indices, item indices, and explicit rating signals.
- **Data Splitting:** The data was divided into three parts: the training set, the validation set, and the test set. Initially, 80% of the data was allocated to the training set, while 20% was reserved for the validation and test sets. The remaining 20% was then evenly split between the validation and test sets, with 50% assigned to each.
- **Preparation of Model Inputs:** for each interaction sample, the corresponding user index and item index paired with the observed rating value. User and item indices are fed into embedding layers within the NCF architecture, while the rating values serve as supervised regression targets for explicit rating prediction.

3.3. Baseline Models and Proposed Enhanced Architecture

3.3.1 SASRec (Baseline Model)

SASRec is employed as the first baseline model in this study. It is a unidirectional self-attention based sequential recommender that predicts the next item based solely on the left-to-right historical context. The model captures short- and medium-range dependencies in user behavior by applying

self-attention to the user’s past interactions, while enforcing a causal mask to prevent information leakage from future positions. Due to its simplicity, efficiency, and strong performance on sequential recommendation benchmarks, SASRec serves as an appropriate baseline for evaluating the incremental improvements introduced by BERT-based architectures.

3.3.2 BERT4Rec (Baseline Model)

BERT4Rec is used as the main baseline model for sequential recommendation. Unlike SASRec, BERT4Rec applies bidirectional Transformer encoders, allowing the model to learn contextual item representations from both left and right positions within the sequence. Training is conducted using a Cloze-style masked item prediction objective, where a subset of items in the sequence is masked, and the model is tasked with reconstructing them based on the surrounding context. This bidirectional training strategy enables BERT4Rec to capture long-range dependencies and complex interaction patterns more effectively than unidirectional approaches. In this study, the original BERT4Rec architecture is used as a strong baseline against which the proposed enhanced BERT4Rec model is compared.

3.3.3 Proposed Optimized BERT4Rec

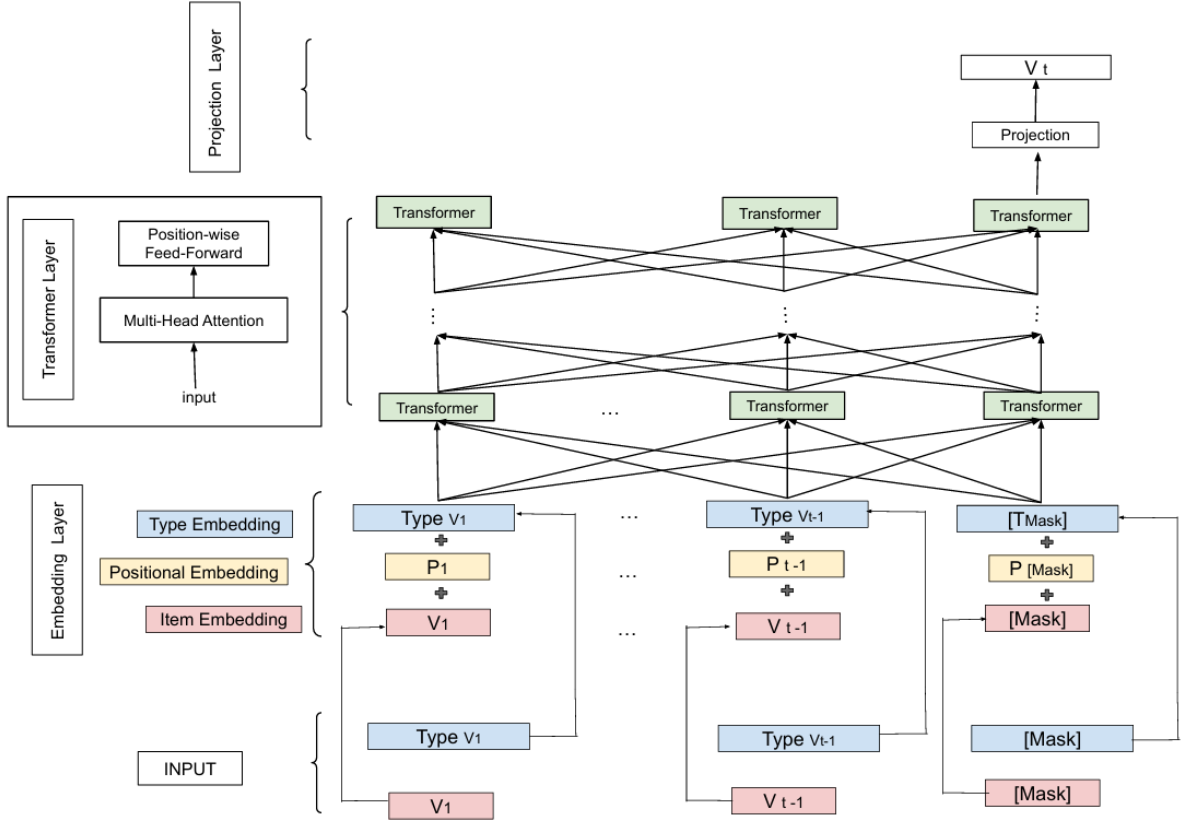


Figure 3. 3: Architecture of the Proposed Optimized BERT4Rec model.

In this section, we describe the architecture of the proposed Optimized BERT4Rec model, which extends the original BERT4Rec framework by integrating auxiliary product-type information to enhance sequential recommendation performance. While the core Transformer architecture with multi-head self-attention is retained, the proposed model introduces a ProductType Embedding and a synchronized masking strategy to better capture categorical context and prevent information leakage during training.

- **Embedding Layer:** the Embedding Layer is responsible for converting the input data into dense vector representations suitable for processing by the model. It consists of three components: Item Embedding, where the ProductId is encoded into a dense vector that captures the semantic properties of each product; Positional Embedding, which ensures that

the model learns the relative position of each item in the interaction sequence, preserving its temporal nature; and ProductType Embedding, which encodes the categorical type of each product as auxiliary contextual information. These embeddings are combined through summation to form a unified representation that jointly captures item identity, positional order, and category-level context.

- **Transformer Layers:** After the integrated embeddings are generated, they are passed through a stack of Transformer layers composed of multi-head self-attention and position-wise feed-forward networks. The self-attention mechanism allows each item position to attend to all other positions within the sequence, enabling the model to capture rich contextual relationships among items. Through this process, the representation at each position becomes context-aware, encoding information from the entire interaction sequence rather than relying solely on the most recent item. In particular, the contextualized representation at a masked position aggregates relevant signals from surrounding items via self-attention, forming a comprehensive summary of the user's interaction history. This representation serves as the key input for the recommendation stage, as it is subsequently used to predict the most probable item for the masked position. By stacking multiple Transformer blocks, the model progressively refines these context-aware representations, leading to more accurate and robust recommendations.
- **Projection Layer:** the contextualized representations produced by the Transformer layers are projected into the item vocabulary space through a linear projection layer, followed by a SoftMax operation. The model outputs a probability distribution over all candidate items for each masked position. Recommendation is performed by selecting the items with the highest predicted probabilities at the masked position. During inference, the last position of

the interaction sequence is masked, allowing the model to generate top-K recommendations for the next item based on the learned contextual representations.

- **Masking Mechanism:** During inference, the trained model is applied in a masked prediction setting consistent with the BERT-style objective. Specifically, the last position of each user interaction sequence is replaced with a designated mask token, and the corresponding product-type feature is masked accordingly. The contextualized representation at this masked position, produced by the Transformer layers, is then used to predict the most probable next item. This strategy enables the model to leverage the full bidirectional context of the observed interaction history.

By integrating ProductType Embedding into both the representation learning and masking processes, the proposed model extends the original BERT4Rec architecture in a controlled and principled manner. This design enables the model to capture not only item-level sequential dependencies but also category-level semantic coherence, which is particularly beneficial in domains characterized by diverse product interactions.

3.3.4 Neural Collaborative Filtering (NCF)

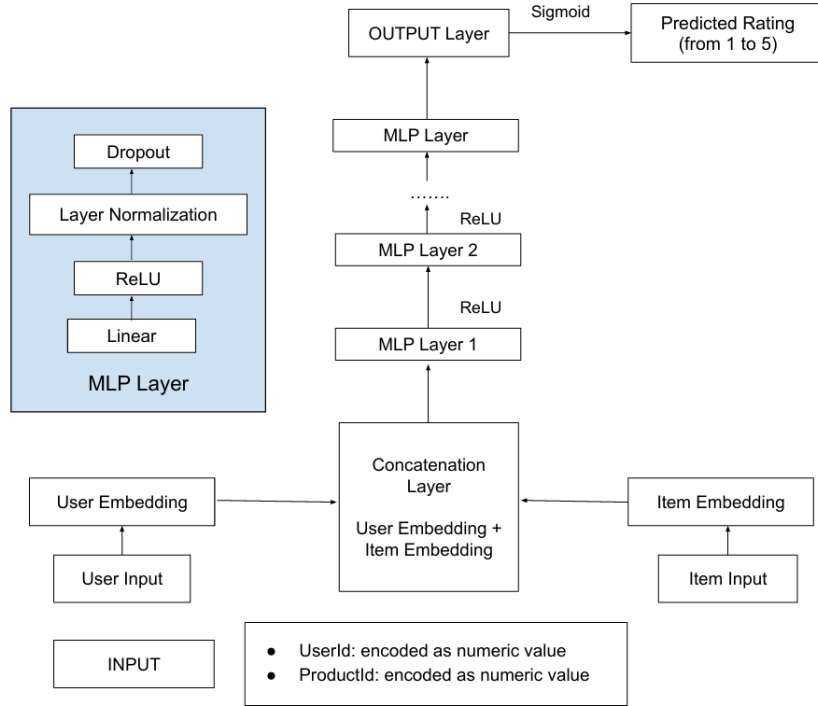


Figure 3. 4: Architecture of the Neural Collaborative Filtering (NCF) model.

The Neural Collaborative Filtering (NCF) model is designed to predict ratings for a given user-item interaction. The architecture of the model follows a structured sequence from input encoding to final rating prediction.

- Input Layer of the Neural Collaborative Filtering (NCF) model is responsible for receiving two key pieces of information: the User ID and the Product ID. The User ID is encoded as a numerical value, serving as a unique identifier for each user in the system. Similarly, the Product ID is encoded as a numerical value, representing the unique products within the recommendation system. This encoding process for both the User ID and Product ID is crucial as it transforms the categorical data into a numerical format, which can then be fed into the neural network. This allows the model to effectively learn patterns and relationships associated with individual users and products.

- Embedding Layer transforms the numerical representations of User ID and Product ID into dense, continuous-valued vectors, known as embeddings. Each user and product is mapped to a vector of fixed dimensionality, which serves as a low-dimensional, dense representation of these entities. User Embedding and Product Embedding are learned through training and capture the underlying patterns and relationships for each user and product based on historical interaction data.
- Concatenation Layer: the embeddings for both users and products are obtained, they are concatenated into a single vector. This concatenation merges the individual representations of the user and the product, capturing the relationship between the two. This process is vital, as it combines both user and product-specific information into one vector that can be processed further in the network.
- Multi-Layer Perceptron (MLP): the concatenated user and product embeddings are passed through several layers of a Multi-Layer Perceptron (MLP). The MLP consists of multiple fully connected layers, each with activation functions (typically ReLU) and dropout for regularization.
- The MLP learns the complex, non-linear interactions between the user and product embeddings. This allows the model to capture more intricate patterns in user preferences and product characteristics beyond simple linear interactions. The MLP layers are followed by a final output layer, which produces a predicted rating value for the user-product pair.
- Output Layer: final output of the MLP layer is a scalar value representing the predicted rating. This rating is a floating-point number, typically between 1 and 5, indicating the predicted score or preference that the user has for the product. The output layer applies to a sigmoid activation function to restrict the range of the predicted rating to a desired range.

3.4. Evaluation Metrics

3.4.1. Performance Evaluation of BERT4Rec

To evaluate the performance of the BERT4Rec model and its baseline models in sequential recommendation tasks, we use two widely recognized evaluation metrics commonly employed in ranking and recommendation systems: NDCG@K (Normalized Discounted Cumulative Gain at K) and Hit@K (Hit Ratio at K). These metrics are used to assess how well the models rank relevant items within the top-K recommendations. The evaluation helps to assess the effectiveness of BERT4Rec as compared to the baseline models in sequential recommendation tasks.

NDCG@K (Normalized Discounted Cumulative Gain): this metric is used to evaluate the ranking quality of the top-K recommended items. It considers both the relevance of the items and their position in the recommendation list. The position of relevant items is important, and NDCG rewards higher-ranked relevant items.

$$NDCG@K = \frac{1}{Z_K} \sum_{i=1}^K \frac{rel(i)}{\log_2(i+1)} \quad [19]$$

where:

- $rel(i)$ is the relevance of the item at position i , which is usually 1 if the item is relevant and 0 if not.
- Z_K is the normalization factor, which ensures that the value of NDCG is between 0 and 1.

Hit@K (Hit Ratio at K): is a commonly used metric for evaluating the accuracy of recommendation models, specifically measuring whether the relevant item (the one the user interacted with or rated) appears in the top-K recommended items. If the relevant item is included in the top-K, it counts as a "hit." This metric directly reflects the ability of the recommendation

model to prioritize relevant items in the recommendation list. A higher Hit@K value indicates a more successful recommendation system in terms of presenting items that are relevant to the user.

$$Hit@K = \frac{1}{|U|} \sum_{u \in U} 1_{\{r_u \leq K\}} \quad [20]$$

Where:

- U is the set of all users in the test set.
- r_u is the rank of the relevant item for user u in the recommendation list.
- $1_{\{r_u \leq K\}}$ is an indicator function that returns 1 if the relevant item is within the top-K recommendations, and 0 otherwise.
- K is the number of recommendations to consider

3.4.2. Performance Evaluation of NCF

The Neural Collaborative Filtering (NCF) model is evaluated based on three key metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). These metrics help to quantify how accurately the model predicts user-item ratings and how close the model's predictions are to the actual ratings.

Mean Squared Error (MSE): measures the average squared difference between the predicted ratings and the true ratings. It is widely used in regression tasks to evaluate the model's prediction accuracy. The lower the MSE, the better the model's predictions.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad [19]$$

where:

- y_i is the true rating of the i -th user-item pair,
- $\hat{y}_i$ is the predicted rating for the same pair,

- N is the number of user-item pairs in the dataset.

Root Mean Squared Error (RMSE): is the square root of the MSE. It provides a more interpretable measure in the same units as the ratings (rather than in squared units). RMSE is useful for understanding the typical magnitude of the error made by the model.

$$RMSE = \sqrt{MSE} \quad [19]$$

Mean Absolute Error (MAE): calculates the average absolute difference between the predicted ratings and the true ratings. Unlike MSE, MAE does not penalize large errors as much, and it gives a more straightforward view of the model's average prediction error.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i| \quad [19]$$

The evaluation metrics chosen for this study - RMSE, MSE, and MAE are selected to provide a comprehensive assessment of the NCF model's performance. RMSE is the primary metric due to its interpretability in the same units as the ratings and its ability to penalize large errors, making it ideal for recommendation tasks. MSE, while less interpretable, is useful for emphasizing large errors during model optimization. Finally, MAE is included for its robustness to outliers and its ability to provide a simple, straightforward measure of the average prediction error. These metrics together offer a balanced evaluation of the model's prediction accuracy and generalization ability.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1. Implementation Details and Experimental Setup

4.1.1. Implementation of the Proposed Optimized BERT4Rec

4.1.1.1. Environment

The models were implemented in Python using the PyTorch deep learning framework. All experiments were conducted in the Google Colaboratory environment, utilizing an NVIDIA T4 GPU with 16 GB of memory and CUDA support for computational acceleration.

4.1.1.2. Train-Test Split Strategy

The dataset was partitioned using the Last-Item-Out (Leave-One-Out) strategy, a standard protocol in sequential recommendation research designed to rigorously evaluate the model's ability to predict future user behavior, this strategy ensures a strict chronological separation between the historical data used for model training and the prospective items reserved for evaluation. Specifically, for every user's complete chronological sequence of interactions, the data was separated as follows: the initial portion of the sequence, encompassing all but the last two items, was designated as the Training Sequence, which provides the requisite historical context for the Masked Item Prediction objective; the second-last item was reserved as the Validation Target, forming a dedicated validation set used to monitor performance and select the most suitable training epoch (checkpoint); and the single final item was held out as the Test Target, which serves as the definitive evaluation benchmark measuring the model's prediction accuracy for the user's next action. This methodical split effectively creates an unbiased prediction scenario in which the model must successfully infer future items based purely on the preceding interactions.

```

train_seq = seq[:-2]
valid_item = seq[-2]
test_item = seq[-1]

user_train[user] = train_seq
types_seq[user] = [self.item_type(i) for i in train_seq]
user_valid[user] = valid_item
user_test[user] = test_item

```

4.1.1.3. Model Configuration

To ensure a fair and transparent comparison, all BERT-based models in this study were trained under a shared configuration. The main architectural and training hyperparameters, which largely follow the settings reported in the original BERT4Rec implementation, are summarised in Table 4.1.

Table 4. 1: Model Configuration and Hyperparameters for BERT-based Models

Parameter	Value	Description
Maximum Sequence Length	40	Interaction sequences were truncated or padded to a fixed length.
Hidden Units	256	The dimensionality of the item and positional embeddings was set to 256, defining the size of the vectors propagated through the Transformer blocks.
Number of Attention Heads (H)	2	The Multi-Headed Attention mechanism utilized 2 heads, allowing the model to jointly attend to information from different representation subspaces.
Number of Transformer Layers (L)	2	Number of stacked Transformer blocks in the BERT encoder.

Dropout Rate	0.1	Probability applied throughout the embedding layer, attention mechanism, and feed-forward networks to prevent overfitting.
Learning Rate	0.001	The learning rate used by the Adam optimizer.
Batch Size	256	The batch size used during training.
Number of Epochs	50	The number of complete passes over the training dataset.
Masking Probability	0.4	Crucial for the Masked Item Prediction objective, the probability of an item being randomly masked, replaced.

- Evaluation Details:

The model’s effectiveness in the sequential recommendation task was assessed using a leave-one-out evaluation protocol, which focused on predicting the final item in each user’s sequence. For each test user, a candidate set was constructed that contained the held-out ground-truth item together with $N = 100$ randomly sampled negative items that the user had not previously interacted with. This $N + 1$ candidate pool allowed for the efficient computation of ranking metrics. Normalized Discounted Cumulative Gain at K (NDCG@ K) accounted for the graded relevance of items in a recommendation list by aggregating their gains and discounting their contribution according to their rank positions. In this study, NDCG@5 and NDCG@10 were used to evaluate how well the system ordered items within the top 5 and top 10 recommendations, where larger NDCG values indicated that the ground-truth items were placed nearer the top of the list. Hit Rate at K (Hit@ K)

complemented this by assessing whether the system successfully placed the relevant item within the top-K positions; Hit@5 and Hit@10 were computed as the fraction of users whose held-out target item appeared in the top-K recommendations, with higher HR values signifying a stronger ability of the model to retrieve relevant items.

- Recommend methods:

During inference, the recommendation process follows a masked prediction paradigm that is fully aligned with the training objective of the model. Specifically, given a user's historical interaction sequence, the item at the final position is replaced with a designated mask token, and the associated auxiliary feature, such as the product-type information, is masked accordingly to prevent information leakage. The masked sequence is processed via a forward pass of the trained Transformer model, in which self-attention mechanisms compute contextualized representations for each position in the sequence. The hidden representation corresponding to the masked position is subsequently extracted, as it encodes the user's dynamic preference conditioned on the entire interaction history. This representation is then projected into the item vocabulary space through a linear transformation followed by a scoring function, producing relevance scores for all candidate items. Based on these predicted scores, items are ranked in descending order, and the top-K items with the highest relevance scores are selected and returned as the final recommendation results for the user.

4.1.2. NCF Implementation

- Environment

The NCF model was also implemented in Python using the PyTorch framework and trained in Google Colaboratory on an NVIDIA T4 GPU (16 GB) with CUDA-enabled hardware acceleration.

- Train Test split

The NCF model was trained on explicit feedback data (ratings) that was partitioned using a three-way random hold-out scheme. The full set of user–item interactions was randomly divided into three mutually exclusive subsets:

Training Set: 80% of the interactions were allocated to the training set. This subset was used to optimize the model parameters and learn the latent representations of users and items.

Validation Set: 10% of the interactions were assigned to a validation set. This set was used during training to monitor the model’s generalization performance, control the learning-rate scheduler, and apply early stopping based on the validation RMSE.

Test Set: the remaining 10% of the interactions were reserved as a held-out test set, which was only used once for the final evaluation to provide an unbiased estimate of the model’s performance on entirely unseen rating predictions.

```
train_idx, temp_idx = train_test_split(
    indices,
    test_size=0.2,
    random_state=42,
    shuffle=True
)
val_idx, test_idx = train_test_split(
    temp_idx,
    test_size=0.5,
    random_state=42,
    shuffle=True
)
```

- Model Configuration

Table 4. 2: Model Configuration and Hyperparameters for NCF Models

Parameter	Value	Description
Embedding Dimension	16	The size of the latent vector space for both user and item embeddings.
MLP Hidden Dimensions	[64,32]	Defines the structure of the MLP layers, starting with 64 units followed by 32 units, learning non-linear interactions.
Activation Function	ReLU	Rectified Linear Unit, used in all hidden layers of the MLP.
Normalization	LayerNorm	Applied to embeddings and hidden layers to stabilize the training process.
Loss Function	MSE Loss	Mean Squared Error, standard for continuous rating prediction tasks.
Optimizer	Adam	Used for stochastic optimization.
Learning Rate (LR)	0.001	Initial learning rate.
Weight Decay	1e-4	L2 regularization applied to prevent overfitting.
Dropout	0.4	Applied within the MLP layers for regularization.
Batch Size	256	The number of samples processed per training step.
Epochs	100	Maximum number of training epochs (subject to Early Stopping).

- Evaluation Details

The performance of the Neural Collaborative Filtering (NCF) model on the explicit rating prediction task was evaluated on a held-out test set that contained 10% of all ratings and was kept completely separate from both the training and validation splits. During training, an additional 10% validation set was used to monitor generalization, regulate the learning-rate scheduler, and apply early stopping based on the validation RMSE. To quantify prediction errors, three standard regression metrics were employed: Mean Squared Error (MSE), which penalized large deviations between predicted and true ratings; Mean Absolute Error (MAE), which measured the average absolute difference; and Root Mean Squared Error (RMSE), which served as the primary metric because its unit aligned directly with the original rating scale. For all three metrics, smaller values indicated better predictive performance, with a lower RMSE suggesting that the predicted scores were, on average, closer to the ground-truth ratings.

4.1.3. Demonstration

This subsection presents a demonstration developed to visualize and qualitatively examine the inference behavior of the proposed BERT4Rec-based sequential recommendation model. The demonstration is implemented using the Streamlit framework, which provides a lightweight and interactive interface for exploring recommendation results under user interaction sequences.

The primary purpose of this demonstration is to illustrate how the trained model generates recommendations given a user’s historical interactions, rather than to evaluate performance or simulate a production-level system. All recommendations shown in the interface are produced by a fixed, pre-trained model operating in an offline inference setting. Consequently, this

demonstration should be regarded as an illustrative tool that complements the quantitative evaluation presented in the previous chapter.

The image shows a web interface titled "Recommendation Demonstration Interface" with the subtitle "BERT4Rec + NCF". It is divided into two main sections. The top section, labeled "Demo", contains a "User ID:" input field, a "Top K:" dropdown menu set to "10", an "Item Sequence:" input field, and an "Avoided Items (optional):" input field. A large green button labeled "Get Recommendations & Validate Model" is positioned below these fields. The bottom section, labeled "Search Products & Build Custom Sequence", features a "Search by product type:" input field with the text "lipstick" and a green "Search" button.

Figure 4. 1: Main demonstration interface

Figure 4.1 presents the main interface of the demonstration implemented using Streamlit. The interface integrates user input, historical interaction visualization, Top-K recommendation results generated by the BERT-based model, and auxiliary rating predictions produced by the NCF model, providing a consolidated view of the demonstration workflow.

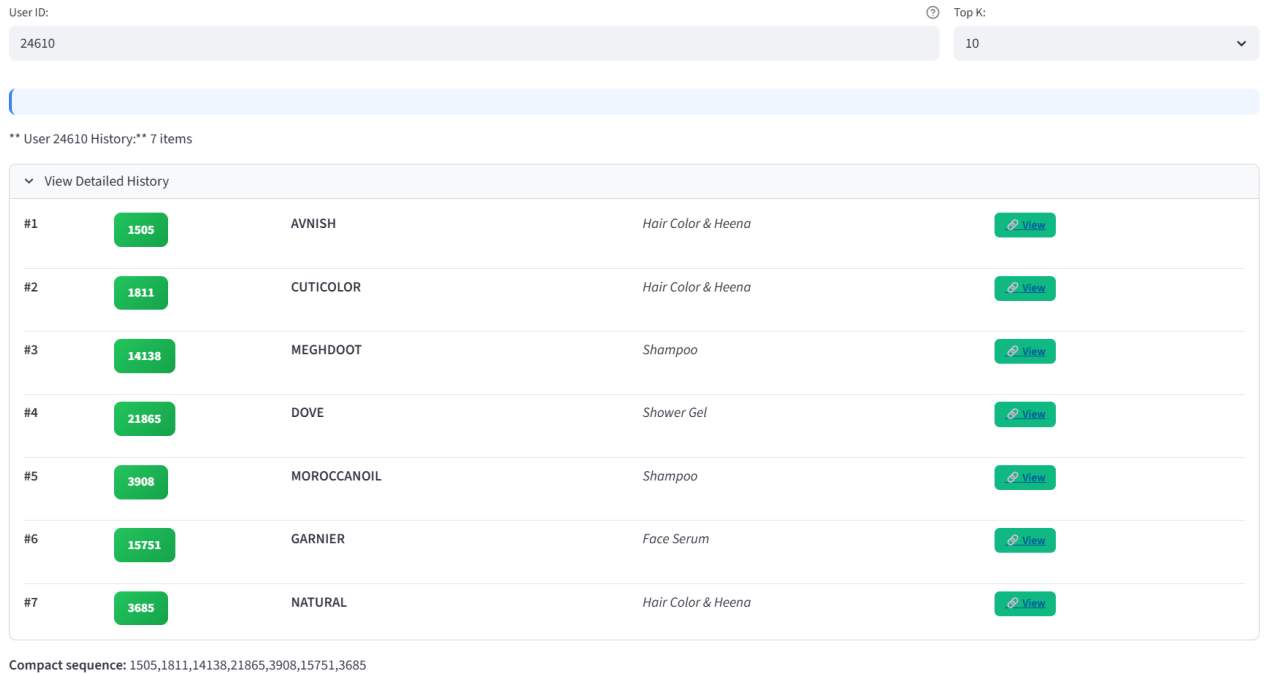


Figure 4. 2: Visualization of the historical interaction sequence for a selected user retrieved

As illustrated in Figure 4.2, the historical interaction sequence of a selected user is retrieved directly from the dataset and displayed in the demonstration interface. The user can specify a user identifier, upon which the corresponding interaction history is loaded and visualized. This visualization provides transparency into the actual behavioral data used as input for the recommendation process.

For each interaction in the sequence, relevant attributes such as item identifiers, associated brands, and product categories are shown. By explicitly displaying the input sequence, the demonstration helps clarify how sequential context is constructed and subsequently consumed by the recommendation model during inference.

Item Sequence:

1505,1811,14138,21865,3908,15751,3685

Avoided Items (optional):

Get Recommendations & Validate Model

Ground Truth (Actual Next Item)

779

Brand: TENNO

Type: Nail Polish

[View Product](#)

Figure 4. 3: Input interaction sequence and ground-truth next item

Top 5 Recommendations

RANK	ID	RATING	NCF RATING	BRAND	PRODUCT TYPE	ACTION
#1	2773	★★★★★ 5.0/5.0	★★★★☆ 4.66/5.0	NIVEA	Face Wash & Cleansers	View
#2	1297	★★★★★ 5.0/5.0	★★★★☆ 4.56/5.0	MCAFFEINE	Shower Gel	View
#3	779 GROUND TRUTH	★★★★★ 5.0/5.0	★★★★☆ 4.7/5.0	TENNO	Nail Polish	View
#4	175	★★★★★ 5.0/5.0	★★★★☆ 4.56/5.0	LA SHIELD	Sunscreen	View
#5	808	★★★★☆ 4.0/5.0	★★★★☆ 4.72/5.0	CLEANSENSE	Shower Gel	View

Figure 4. 4: Top-K recommendation results

As illustrated in Figures 4.3 and 4.4, given an input interaction sequence, the final position in the sequence is replaced with a designated mask token and passed through the trained BERT-based sequential recommendation model to generate Top-K recommendation candidates. The model produces relevance scores for all candidate items, which are subsequently ranked in descending order to form the final Top-K recommendation list.

For demonstration purposes, the ground-truth next item from the dataset is explicitly displayed alongside the recommendation results, as shown in Figure 4.3. This design enables a qualitative

inspection of whether the true next interaction appears within the Top-K recommendations generated by the model. In the illustrated example, the ground-truth item is ranked among the recommended candidates, serving as a representative test case that reflects the inference behavior of the trained model.

Figure 4.4 presents the detailed Top-K recommendation table, including item identifiers, associated brands, product categories, and predicted relevance scores. In addition, the table displays auxiliary rating predictions obtained from an independently trained Neural Collaborative Filtering (NCF) model. These NCF rating scores are computed post hoc for each recommended user–item pair and are shown solely for interpretability purposes. Importantly, the NCF predictions do not affect the ranking or selection of the Top-K recommendations and are not used for re-ranking or candidate refinement.

Search Products & Build Custom Sequence

Search by product type:

Shampoo

Search

Found 2090 products matching 'Shampoo'

12

DOVE

Shampoo

View

Add

Avoid

14

MOROCCANOIL

Shampoo

View

Add

Avoid

18

SUNSILK

Shampoo

View

Add

Avoid

Figure 4. 5: Product search interface for adding items to a custom interaction sequence

Compact sequence: 1505,1811,14138,21865,3908,15751,3685

Item Sequence:

1505,1811,14138,21865,3908,15751,3685,779,353,210,115,12

Avoided Items (optional):

Get Recommendations & Validate Model

Figure 4. 6: Updated interaction sequence after appending new items

Top 5 Recommendations

RANK	ID	RATING	NCF RATING	BRAND	PRODUCT TYPE	ACTION
#1	693	★★★★★ 5.0/5.0	★★★★☆ 4.75/5.0	LOREAL	Shampoo	View
#2	107	★★★★★ 5.0/5.0	★★★★☆ 4.72/5.0	MYGLAMM	Nail Polish	View
#3	2773	★★★★★ 5.0/5.0	★★★★☆ 4.66/5.0	NIVEA	Face Wash & Cleansers	View
#4	1297	★★★★★ 5.0/5.0	★★★★☆ 4.56/5.0	MCAFFEINE	Shower Gel	View
#5	84	★★★★★ 5.0/5.0	★★★★☆ 4.72/5.0	MCAFFEINE	Shower Gel	View

Figure 4. 7: Top-K recommendations generated from the modified sequence

In addition to visualizing historical user sequences, the demonstration supports the dynamic construction and modification of custom interaction sequences. As shown in Figures 4.5–4.7, users can search items by product type and append selected items to an existing sequence while preserving the same user identifier. This design allows controlled manipulation of the input sequence.

After modifying the sequence, the recommendation model is re-invoked using the updated interaction history, resulting in a different Top-K recommendation list. The observed changes in the recommendation output reflect the model’s sensitivity to sequential context, particularly the

influence of recently appended items. This behavior is consistent with the underlying masked self-attention mechanism, which assigns varying importance to items based on their relative positions and contextual relevance within the sequence. Importantly, this dynamic behavior is achieved during inference only; the underlying model parameters remain fixed. The demonstration therefore provides an intuitive understanding of how the Transformer-based architecture captures dependencies within user behavior sequences.

4.2. Results

4.2.1. Results of Sequential Recommendation Models

The evaluation results of the three sequential recommendation models - SASRec, BERT4Rec, and the optimized version, BERT4Rec+ - are presented in Table 4.3. The models are compared using two commonly used metrics for ranking performance in recommendation systems: NDCG (Normalized Discounted Cumulative Gain) and HIT Rate (HR), evaluated at both $K = 5$ and $K = 10$. These metrics offer a comprehensive assessment of the models' ability to rank relevant items higher in the recommendation list, with a focus on how well the models perform in the top-k positions.

Table 4. 3: Performance Comparison of Sequential Models

Metrics	SASRec	BERT4Rec	BERT4Rec+
NDCG@5	0.1786	0.1817	0.2219
HIT@5	0.2646	0.2710	0.3094
NDCG@10	0.2160	0.2233	0.2560
HIT@10	0.3811	0.3997	0.4157

As a baseline sequential recommendation model, SASRec demonstrates competitive performance; however, its effectiveness is consistently surpassed by BERT4Rec across all reported metrics. In particular, BERT4Rec shows clear improvements over SASRec in both NDCG@5 and HIT@5, indicating a stronger ability to rank relevant items at higher positions in the recommendation list.

Specifically, BERT4Rec achieves an NDCG@5 score of 0.1817 compared to 0.1786 for SASRec, and a HIT@5 score of 0.2710 versus 0.2646. Similar performance gains are observed at a larger cut-off, where BERT4Rec outperforms SASRec on NDCG@10 (0.2233 vs. 0.2160) and HIT@10 (0.3997 vs. 0.3811).

These improvements suggest that the Transformer-based architecture of BERT4Rec, leveraging self-attention to model bidirectional dependencies within user interaction sequences, is more effective at capturing complex sequential patterns than the unidirectional design adopted by SASRec.

Table 4. 4: Performance Improvement of BERT4Rec+ over BERT4Rec

Metrics	BERT4Rec	BERT4Rec+	Improvement
NDCG@5	0.1817	0.2219	22.12%
HIT@5	0.2710	0.3094	14.17%
NDCG@10	0.2233	0.2560	14.64%
HIT@10	0.3997	0.4157	4.00%

Table 4.4 summarizes the relative performance improvements of BERT4Rec+ over the original BERT4Rec model across multiple evaluation metrics, including NDCG@5, HIT@5, NDCG@10, and HIT@10. These metrics evaluate the model’s ability to rank relevant items within the top-k recommendation positions.

The results indicate consistent and meaningful improvements across all evaluation criteria. In particular, BERT4Rec+ achieves a 22.12% increase in NDCG@5 and a 14.17% improvement in HIT@5, demonstrating a substantially enhanced ability to prioritize relevant items at the top of the recommendation list. Performance gains are also observed at a larger cutoff, with improvements of 14.64% in NDCG@10 and 4.00% in HIT@10.

These improvements highlight the effectiveness of the modifications introduced in BERT4Rec+, most notably the incorporation of ProductType Embedding. By explicitly modeling product category information, the model is able to capture richer contextual signals that complement user interaction sequences. This additional semantic context allows BERT4Rec+ to better distinguish between candidate items during ranking, particularly in scenarios where users interact with products across diverse categories.

The comparison demonstrates that BERT4Rec+ consistently outperforms both SASRec and the original BERT4Rec model across all evaluated metrics. This consistent improvement indicates that the proposed model is more effective at capturing sequential user preferences and ranking relevant items in the top-K recommendation list. By integrating side information, BERT4Rec+ is able to incorporate additional item-level categorical context into the sequential modeling process, allowing the model to better distinguish between items with similar interaction patterns but different semantic characteristics. Consequently, the generated recommendations are more accurate and context-aware, particularly in scenarios where users interact with products across diverse categories. These results highlight the importance of incorporating item-level categorical features

in sequential recommendation systems and confirm the effectiveness of the proposed BERT4Rec+ model as an enhanced solution for sequential recommendation tasks.

4.2.2. Results of the Neural Collaborative Filtering Model

Table 4. 5: Performance Evaluation of the Neural Collaborative Filtering Model

	MSE	RMSE	MAE
Value	1.2759	1.1295	0.8506

Table 4.4 reports the performance of the Neural Collaborative Filtering model on the held-out test set. The model obtains an MSE of 1.2759, an RMSE of 1.1295, and an MAE of 0.8606 on the 1-5 rating scale. Because RMSE is expressed in the same units as the original ratings, the predicted scores differ from the true scores by roughly one rating point on average. Similarly, the MAE below 1.0 shows that, on average, the absolute deviation between the predicted and observed ratings is slightly less than one star. In other words, the model is generally able to place items in the correct region of the rating scale rather than producing completely mismatched scores. Overall, these results suggest that the NCF model produces reasonably calibrated rating estimates that are sufficient to enrich the recommended products with an interpretable notion of predicted user satisfaction.

CHAPTER 5: DISCUSSION AND EVALUATION

5.1. Discussion

In this study, we proposed an enhanced version of the BERT4Rec model, referred to as BERT4Rec+, with the objective of improving sequential recommendation performance in the beauty product domain. By incorporating additional product-type embeddings and fine-tuning key hyperparameters, BERT4Rec+ consistently demonstrated superior performance compared to the baseline models, SASRec and the original BERT4Rec. The experimental results show notable improvements across all evaluated metrics, confirming the effectiveness of the proposed enhancements.

In particular, BERT4Rec+ achieves substantial gains in NDCG@5 and HIT@5, indicating a stronger ability to rank relevant items at the very top of the recommendation list and to ensure that the ground-truth items appear within the top five recommendations. Improvements are also observed in NDCG@10 and HIT@10, suggesting that the model maintains better ranking quality even when considering a broader recommendation scope. These results highlight the benefit of enriching the input representation with product category information, which allows the model to better capture semantic relationships among items and align recommendations with user preferences across diverse product types.

The findings confirm that BERT4Rec+ represents a meaningful enhancement over its predecessors. By leveraging additional contextual information, the model improves its ability to model sequential user behavior and to generate more accurate and context-aware recommendations. Such improvements are particularly valuable in domains characterized by high product diversity, where understanding both sequential patterns and item semantics is essential for delivering relevant recommendations.

In contrast, the Neural Collaborative Filtering (NCF) model focuses on predicting explicit user–item rating scores and excels in minimizing prediction errors, as reflected by its MSE, MAE, and RMSE values. However, NCF does not explicitly model sequential dependencies or temporal patterns in user interactions, as each user–item pair is treated independently. As a result, NCF is less suitable for next-item recommendation tasks that rely heavily on sequential context.

Despite this limitation, NCF plays a complementary role when used alongside BERT4Rec+. While BERT4Rec+ is responsible for generating Top-K candidate items based on sequential relevance, NCF provides an additional layer of interpretation by estimating the expected rating for each recommended item. In this study, the predicted ratings from NCF are used solely for interpretability and qualitative inspection, rather than for re-ranking or influencing the recommendation results. This combination illustrates how sequence-aware models and rating-based collaborative filtering models can offer complementary perspectives on user preference modeling, enhancing the overall understanding of recommendation outputs without altering the underlying ranking mechanism.

5.2. Comparison

The performance of BERT4Rec+ is compared to two baseline models: SASRec and the original BERT4Rec. These models differ in their architecture, feature inputs, and training strategies, which have a direct impact on their recommendation performance. SASRec is a unidirectional self-attention model, while BERT4Rec incorporates bidirectional Transformer encoders, allowing it to capture sequential dependencies in both directions. BERT4Rec+ further extends BERT4Rec by adding product-type embeddings to enhance item representation, specifically tailored to the domain of beauty products.

The key differences between the models lie in their handling of user interactions and item features. SASRec, a simpler model, uses a unidirectional self-attention mechanism, meaning it only considers the past interactions of users without any context from future items. BERT4Rec, in contrast, utilizes a bidirectional Transformer, enabling it to model both past and future interactions, thus capturing long-range dependencies and more complex user behavior. BERT4Rec+ builds on this by integrating ProductType Embedding, which incorporates additional categorical information (such as product type) into the item representation, enhancing the model’s ability to distinguish between different products.

In terms of performance, BERT4Rec+ outperforms both SASRec and BERT4Rec across various metrics. The inclusion of product-type embeddings allows BERT4Rec+ to improve its ranking and recommendation accuracy. This enhanced model also demonstrates better coverage, showing its ability to rank and include more relevant items within the top-K recommendations. The results confirm that BERT4Rec+ provides a more robust solution for sequential recommendation tasks by capturing both temporal and categorical information.

5.3. Evaluation

The evaluation of BERT4Rec+ and NCF focuses on their effectiveness, practical applicability, and reliability within the scope of sequential recommendation tasks. BERT4Rec+ demonstrates strong ranking performance, particularly in Top-K recommendation scenarios, highlighting its suitability for modeling sequential user behavior. In contrast, NCF specializes in personalized rating prediction, which provides an additional layer of interpretability by estimating users’ expected preferences for recommended items.

From an application perspective, BERT4Rec+ is well suited to recommendation scenarios characterized by high product diversity, such as e-commerce environments. By incorporating

product-type embeddings, the model leverages both historical interaction patterns and item-level categorical information, enabling more context-aware and personalized recommendations. Although NCF is not designed to model sequential dependencies, it complements BERT4Rec+ by offering explicit rating estimates that can assist users in interpreting and comparing recommended items.

Regarding model reliability, BERT4Rec+ demonstrates stable performance on the Amazon Beauty dataset used in this study. However, its generalization to other domains or datasets may require further empirical validation and domain-specific fine-tuning. Moreover, the computational overhead introduced by the Transformer-based architecture could present challenges in large-scale or latency-sensitive applications. These considerations suggest that while BERT4Rec+ is effective within the experimental setting, additional optimization and evaluation would be necessary for broader deployment.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1. Conclusion

In conclusion, this study successfully developed and evaluated an enhanced sequential recommendation framework by leveraging the strengths of two complementary deep learning models: BERT4Rec for sequence-aware recommendation and Neural Collaborative Filtering (NCF) for explicit rating prediction. The experimental results validate the hypothesis that Transformer-based architectures are highly effective in modeling complex, non-linear dependencies in user interaction sequences, with BERT4Rec-based models consistently outperforming traditional sequential baselines such as SASRec across all key ranking metrics, including NDCG@K and HIT@K.

A primary contribution of this work is the integration of Product Type Embedding into the BERT4Rec architecture, resulting in the proposed BERT4Rec+ model. This enhancement enables the model to incorporate item-level categorical information alongside sequential interaction data, thereby improving its ability to contextualize items within user behavior sequences. Empirical evaluations demonstrate that this modification leads to substantial improvements in Top-K recommendation quality, highlighting the importance of enriching self-attention mechanisms with explicit item-side information to support more accurate and context-aware ranking decisions.

In addition to model development and evaluation, this study presents a lightweight demonstration interface implemented using Streamlit to visualize the inference behavior of the proposed models. The demonstration illustrates how BERT4Rec+ generates sequential recommendations and how NCF provides complementary rating predictions for the recommended items. Importantly, the NCF model is used solely for interpretability purposes and does not influence the ranking or selection of Top-K recommendations. This design showcases how

sequence-aware recommendation and rating prediction models can offer complementary perspectives on user preferences without requiring joint training or integrated ranking.

Despite the promising results, several limitations remain. The Transformer-based architecture of BERT4Rec+ introduces higher computational costs, which may pose challenges for large-scale or latency-sensitive applications. Moreover, the generalization of the proposed model to other domains beyond the Amazon Beauty dataset requires further empirical validation. While NCF demonstrates strong performance in rating prediction, its lack of sequential modeling limits its applicability to ranking-oriented tasks. Addressing these limitations, as well as exploring improvements in scalability, efficiency, and adaptive recommendation strategies, constitutes an important direction for future work.

However, there are still some limitations that need to be addressed. BERT4Rec+ is computationally expensive, especially in large-scale applications, and its ability to generalize across different domains needs further testing. NCF, while effective in rating prediction, lacks the ability to rank items, which limits its performance in scenarios requiring precise ranking. These limitations, along with potential improvements in real-time adaptability and model efficiency, will be discussed in Future Work, where opportunities for enhancing scalability, accuracy, and real-time performance will be explored.

6.2. Future Work

While the proposed BERT4Rec+ and NCF models have shown promising results, several areas of improvement and future research directions remain to further enhance the performance and applicability of the recommendation system.

Future work on the proposed recommendation system can focus on improving model efficiency and scalability, particularly for large-scale applications. Optimizing BERT4Rec+ through

techniques like pruning, knowledge distillation, and quantization could significantly reduce computational demands while maintaining performance. Additionally, enhancing the real-time adaptability of the model is crucial, as currently, the system operates offline. Implementing online learning or reinforcement learning techniques could enable the system to dynamically adjust recommendations based on real-time user interactions, ensuring more personalized and up-to-date suggestions. Another promising direction is integrating multi-modal data (such as images, reviews, or social media mentions) to improve recommendation accuracy. By incorporating both visual and textual information, the model can provide richer and more context-aware recommendations.

REFERENCES

- [1] S. Raza *et al.*, "A comprehensive review of recommender systems: Transitioning from theory to practice," *Computer Science Review*, vol. 55, p. 100849, Feb. 2025, doi: 10.1016/j.cosrev.2025.100849.
- [2] D. Das, L. Sahoo, and S. Datta, "A survey on recommendation system," *International Journal of Computer Applications*, vol. 160, no. 7, pp. 6–10, Feb. 2017, doi: 10.5120/ijca2017913081.
- [3] C. C. Aggarwal, *Recommender Systems: The Textbook*. Cham, Switzerland: Springer, 2016.
- [4] L.-W. Pan, W.-K. Pan, M.-Y. Wei, H. Yin, and Z. Ming, "A survey on sequential recommendation," *Frontiers of Computer Science*, vol. 20, no. 3, Art. no. 2003606, Oct. 28, 2025, doi: 10.1007/s11704-025-41329-w.
- [5] S. Rendle, C. Freudenthaler, and L. Schmidt-Thieme, "Factorizing personalized Markov chains for next-basket recommendation," in *Proc. of the 19th International World Wide Web Conference (WWW '10)*, Raleigh, NC, USA, Apr. 2010, pp. 811–820, doi: 10.1145/1772690.1772767.
- [6] T. Donkers, B. Loepp, and J. Ziegler, "Sequential user-based recurrent neural network recommendations," in **Proc. of the 11th ACM Conference on Recommender Systems*, Como, Italy, Aug. 27–31, 2017, pp. 152–160, doi:10.1145/3109859.3109877.
- [7] B. Hidasi and A. Karatzoglou, "Recurrent neural networks with Top-k gains for session-based recommendations," in **Proc. of the 27th ACM International Conference on Information and Knowledge Management (CIKM '18)**, Torino, Italy, Oct. 22–26, 2018, pp. 843–852, doi:10.1145/3269206.3271761.
- [8] J. Tang and K. Wang, "Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding," in **Proc. of the 11th ACM International Conference on Web Search and Data Mining*, Marina Del Rey, CA, USA, Feb. 5–9, 2018, pp. 565–573, doi:10.1145/3159652.3159656.

- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in **Proc. of the 31st Conference on Neural Information Processing Systems (NeurIPS ’17)**, Long Beach, CA, USA, Dec. 4–9, 2017, pp. 5998–6008.
- [10] W.-C. Kang and J. McAuley, “Self-attentive sequential recommendation,” **arXiv preprint arXiv:1808.09781**, Aug. 20, 2018.
- [11] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, “BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer,” in **Proc. of the 28th ACM International Conference on Information and Knowledge Management (CIKM ’19)**, Beijing, China, Nov. 3–7, 2019, pp. 1441–1450, doi:10.1145/3357384.3357895.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in **Proc. of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT ’19)**, Minneapolis, MN, USA, Jun. 2019, doi:10.18653/v1/N19-1423.
- [13] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” **arXiv preprint arXiv:1607.06450**, Jul. 2016.
- [14] J. Li, Y. Wang, and J. McAuley, "Time Interval Aware Self-Attention for Sequential Recommendation," *Proc. of the 13th Int. Conf. on Web Search and Data Mining (WSDM '20)*, Houston, TX, USA, Feb. 3-7, 2020, pp. 322-330, doi:10.1145/3159652.3159745.
- [15] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, “Neural collaborative filtering,” in **Proc. of the 26th International Conference on Neural Information Processing Systems (NeurIPS ’17)**, Perth, Australia, Apr. 3–7, 2017, pp. 173–182.
- [16] H. Zhang, F. Shen, W. Liu, X. He, H. Luan, and T.-S. Chua, “Discrete collaborative filtering,” in **Proc. of the 39th International ACM SIGIR Conference on Research and Development in*

Information Retrieval (SIGIR '16)*, Pisa, Italy, Jul. 17–21, 2016, pp. 325–334, doi:10.1145/2911451.2911512.

[17] X. He, H. Zhang, M.-Y. Kan, and T.-S. Chua, “Fast matrix factorization for online recommendation with implicit feedback,” in *Proc. of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '16)*, Pisa, Italy, Jul. 17–21, 2016, pp. 549–558, doi:10.1145/2911451.2911537.

[18] S. Rendle, W. Kriche, L. Zhang, and J. Anderson, “Neural collaborative filtering vs. matrix factorization revisited,” in *Proc. of the 14th ACM Conference on Recommender Systems (RecSys '20)*, Virtual Event, Brazil, Sept. 22–26, 2020, pp. 1–9, doi:10.1145/3383313.3412247.

[19] A. Jadon and A. Patil, “A comprehensive survey of evaluation techniques for recommendation systems,” *arXiv preprint arXiv:2312.16015*, Jan. 12, 2024.

[20] A. Prakash, D. Bermperidis, and S. Chennu, “Evaluating performance and bias of negative sampling in large-scale sequential recommendation models,” *arXiv preprint arXiv:2410.17276*, Oct. 8, 2024.

[X] P. Satra, “Amazon Beauty Products,” Kaggle Dataset, 2021. [Online]. Available: <https://www.kaggle.com/datasets/satrapankti/amazon-beauty-product-recommendation>